CONFIDENTIAL
AUTHORIZED USE ONLY

RED TEAM
OPERATIONS

DVWA

FULL PENETRATION TEST REPORT

ASSESS. EXPLOIT. ELEVATE. OWN.

RECONNAISSANCE
Identify. Map. Target.

VULNERABILITY ANALYSIS
Discover. Analyze. Prioritize.

EXPLOITATION
Deliver. Exploit. Break In.

PRIVILEGE ESCALATION
Climb. Escalate. Control.

MAINTAIN ACCESS
Persist. Pivot. Stay Hidden.

REPORTING & REMEDIATION
Document. Protect. Improve.



SECURE.
TEST.
REPEAT.
IMPROVE.

SECURING TODAY
PROTECTING TOMORROW

DATE: 25 MAY 2025
VERSION: 1.0

Summary....2
Overview....3
Results....3
Impact....4
Remediation advises....4
Vulnerability writeups....4
Title....4
Risk rating:....4
Summary:....4
Background:....5
Technical analysis and Evidence:....5
Impact....6
Remediation advises:....6
Title:....6
Risk rating:....6
Summary:....6
Background:....6
Technical details and Evidence:....7
Impact:....7
Remediation advises:....7
Title....8
Risk rating....8
Summary:....8
Background:....8
Technical findings and Evidence:....8
Impact....9
Remediation Advice....9
Title:....9
Risk rating:....9
Summary:....9
Background:....9

Technical Details and Evidence 9
Impact 10
Remediation Advice 11
Title: 11
Risk rating: 11
Summary: 11
Background: 11
Impact: 12
Remediation advises: 12
Title: 12
Risk rating: 12
Summary: 12
Background: 13
Technical Details and Evidence: 13
Impact: 13
Remediation advises: 14
Title: 14
Risk rating: 14
Summary: 14
Background: 14
Technical Details and Evidence: 14
Impact: 16
Remediation advise: 16

Summary

Overview

This assessment was conducted on DVWA to explore common web application vulnerabilities and basic exploitation techniques.

- Brute Force: Used Hydra to automate passwords against the login form. The application's error messages helped identify failed logins and validate username existence.
- Command Injection: Demonstrated how unsensitized input can allow execution of system commands on the server.
- File Inclusion (LFI)**:** Exploited input handling to access sensitive system files such as /etc/passwd, confirming improper file validation.
- File Upload: Uploaded malicious files containing JavaScript payloads, leading to potential XSS execution.
- SQL Injection**:** Tested login parameters using Burp Suite and SQL payloads such as ' or 1=1 #, indicating weak query handling.
- DOM XSS: Identified client-side vulnerability where untrusted input was processed by JavaScript and injected into the DOM.
- Client-Side Logic Flaw: Exploited weak token validation involving ROT13 encoding and MD5 hashing to bypass authentication logic.

Results

The DVWA environment revealed multiple common web application vulnerabilities:

- Brute Force: Authentication was susceptible to automated password guessing using Hydra due to weak failure handling.
- Command Injection: User input was not properly sanitized, allowing execution of system-level commands.
- File Inclusion (LFI)**:** Sensitive system files such as /etc/passwd could be accessed through manipulated input.
- File Upload**:** Improper file validation allows potentially malicious files to be uploaded.
- SQL Injection: Login parameters were vulnerable to manipulation using basic SQL payloads.
- DOM XSS**:** Client-side JavaScript processed untrusted input, leading to script execution in the browser.

- Client-Side Logic Flaw**:** Weak token validation (ROT13 + MD5) was bypassed by reproducing the expected token.

Impact

The vulnerabilities identified in the DVWA environment demonstrate how insecure web application design can lead to severe security risks. Exploitable weaknesses such as SQL Injection, Command Injection, and File Inclusion can allow an attacker to gain unauthorized access to sensitive data, execute arbitrary system commands, and potentially take full control of the underlying server. Brute force attacks further expose weak authentication mechanisms, enabling credential compromise when proper rate limiting or account lockout controls are absent. Client-side issues such as DOM-based XSS and insecure file upload handling can be used to execute malicious scripts in users' browsers, leading to session hijacking, data theft, or defacement. Overall, these vulnerabilities highlight critical failures in input validation, authentication, and secure coding practices, which if present in a real-world application could result in data breaches, system compromise, and loss of user trust.

Remediation advises

To secure the application, all user inputs should be validated and sanitized, and SQL queries must use prepared statements to prevent injection attacks. Implement rate limiting and account lockout to protect against brute-force attacks. File uploads should be restricted by type and safely stored outside the web root. File inclusion risks can be reduced by avoiding direct user input in file paths and using allow-lists. For XSS, ensure proper output encoding and avoid unsafe DOM manipulation methods like inner HTML. Overall, strong server-side validation and secure coding practices are essential to prevent exploitation.

Vulnerability writeups

Title: Weak authentication led to an authorized login (T1110.001 Password Guessing)

Risk rating: high (8.0)

Summary:

A brute-force attack was conducted against the DVWA authentication mechanism using Hydra to evaluate the strength of login security controls. The testing targeted the admin account by systematically attempting multiple password combinations from a wordlist.

The attack leveraged the application's response message ("Username and/or password incorrect") to distinguish failed authentication attempts and assess the validity of credentials. The results demonstrated that the authentication system relies on predictable error feedback and lacks adequate protections such as rate limiting, account lockout, or login attempt monitoring, making it susceptible to credential guessing attacks.

Background:

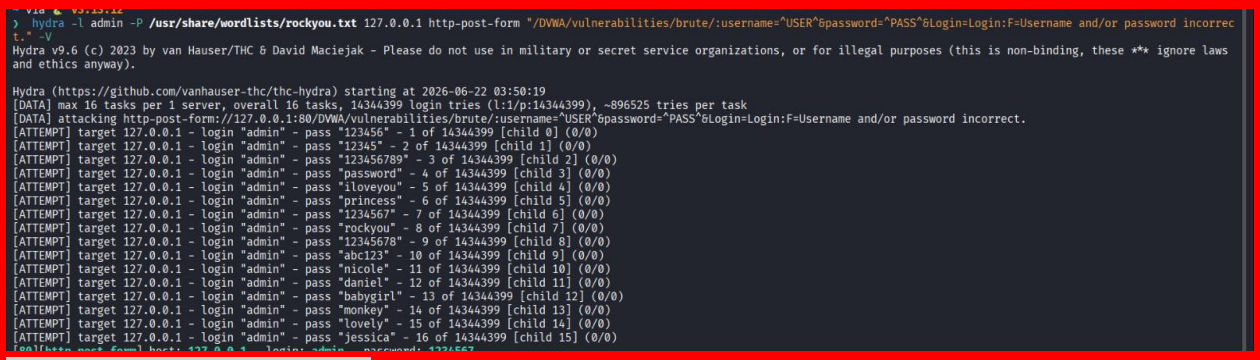
Attackers often target web application logic systems as an initial entry point into a network. By exploiting weak authentication controls, such as simple passwords, missing rate limiting, or informative error messages, an attacker can perform brute-force or password-spraying attacks to gain unauthorized access to user accounts. Once valid credentials are obtained, the attacker may escalate access to sensitive data, compromise user accounts, perform privilege escalation, or move laterally within internal systems. In some cases, compromised administrative accounts can lead to full application or server takeover, allowing data theft, service disruption, or further exploitation of connected systems.

Technical analysis and Evidence:

The web application displays a generic error message, "username or password incorrect," when login attempts fail. While this is a good security practice as it prevents username enumeration, it can still expose the application to password guessing attacks



The username admin was used as the target account during the assessment, and Hydra was employed to perform a brute-force attack against the login functionality. By systematically testing multiple passwords from a wordlist, the correct credentials were eventually identified, confirming that the admin account was valid and active. Successful authentication demonstrated that the application is vulnerable to brute-force attacks due to insufficient protections such as rate limiting or account lockout mechanisms.



The password was found.

Impact

Successful brute-force attacks can lead to unauthorized access to valid user accounts, including administrative accounts. If an attacker gains access, they may be able to view, modify, or delete sensitive data, escalate privileges, and compromise the entire application. The lack of strong authentication controls such as rate limiting or account lockout significantly increases the risk of credential compromise and system abuse.

Remediation advises:

To mitigate brute-force attacks, implement account lockout or temporary suspension after multiple failed login attempts, along with rate limiting to restrict repeated requests from the same IP address. Enforce strong password policies to reduce the risk of guessing successful credentials. Multi-factor authentication (MFA) should be enabled to add an additional layer of security beyond passwords. Additionally, monitoring and logging of failed login attempts should be implemented to detect and respond to suspicious activity early.

Title: Command Injection (T1059) leading to unauthorized system execution and data exposure.

Risk rating: high

Summary:

The attack where an attacker can execute arbitrary system commands through a vulnerable web application. In this assessment, the vulnerability allowed injected commands to be processed by the server, resulting in the exposure of sensitive system information. This demonstrates how improper input validation can lead to unauthorized command execution and potential system compromise.

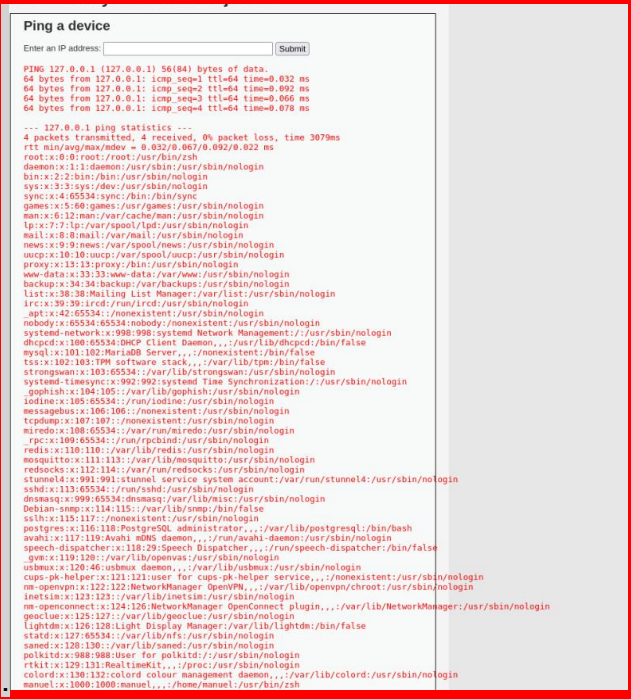
Background:

Command injection allows an attacker to execute operating system commands on a vulnerable server through a web application. This can enable the attacker to gather sensitive system information, read or modify files, create or delete data, and potentially establish persistent access to the system. In more severe cases, it can lead to full server compromise, allowing attackers to pivot into internal networks, deploy malware, or disrupt services. This makes command injection a critical vulnerability, as it directly bridges web application flaws with underlying operating system control.

Technical details and Evidence:

The application allowed user input in the form of an IP address. However, the input field was vulnerable to command injection. By entering 127.0.0.1; cat /etc/passwd, it was possible to execute additional system commands on the server. This resulted in the disclosure of sensitive system information, including user account details from the /etc/

Passwd file.



Impact:

Command Injection can lead to full system compromise, allowing an attacker to execute arbitrary operating system commands. This may result in unauthorized access to sensitive files, exposure of user and system information, data theft, service disruption, and potential lateral movement within the network.

Remediation advises:

To prevent command injection, all user input must be strictly validated and sanitized. Avoid passing user input directly to system commands. Use safe APIs that do not invoke the operating system shell and implement allow-list validation for expected input formats (e.g., valid IP addresses only). Additionally, apply least privilege principles so that the application runs with minimal system permissions.

Title: File inclusion (T1005) that leads to exposure to sensitive information.

Risk rating: High

Summary:

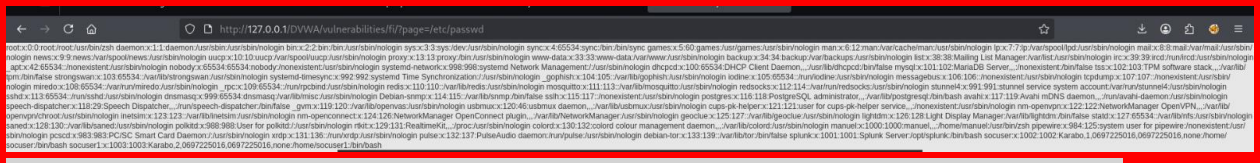
File Inclusion is a web vulnerability that occurs when a PHP application allows user input to control which files are loaded or executed by the server. In this assessment, the URL parameter was manipulated to include system files, and accessing /etc/passwd confirmed a Local File Inclusion (LFI) vulnerability. This exposed sensitive system information, including user account data, due to improper input validation and unsafe file handling.

Background:

Attackers exploit File Inclusion vulnerabilities to read sensitive files on the server, such as configuration files, credentials, and system data. In more advanced cases, LFI can be escalated to Remote Code Execution (RCE) by including malicious files or leveraging log poisoning techniques. This can allow attackers to gain full control over the web server, steal sensitive data, modify application behavior, or use the compromised system as a foothold to attack internal networks.

Technical findings and Evidence:

The application used a URL parameter (page) to determine which content was loaded. During testing, the parameter was manipulated to access unintended files on the server. By modifying the URL and supplying a file path, it was possible to retrieve the contents of sensitive system files, including /etc/passwd. This confirmed the presence of a Local File Inclusion (LFI) vulnerability, allowing unauthorized access to sensitive information stored on the server.



The application returned the contents of the file, exposing system user account

information and demonstrating insufficient validation of user-supplied input used in file inclusion operations.

Impact

The Local File Inclusion (LFI) vulnerability allows an attacker to access sensitive files on the server without authorization. By exploiting this weakness, attackers may obtain system information, configuration files, credentials, and other confidential data. In some cases, LFI can be leveraged to achieve remote code execution, potentially resulting in full server compromise.

Remediation Advice

Implement strict input validation and avoid using user-supplied input to determine file paths. Use an allow-list of approved files that can be accessed by the application and restrict access to sensitive system directories. Additionally, disable unnecessary file inclusion functions, apply the principle of least privilege, and regularly review application code for insecure file handling practices.

Title: File upload ()

Risk rating: Critical

Summary:

A file upload vulnerability was identified within the application. During testing, an HTML file containing a JavaScript payload (<script>alert ("You are hacked") </script>) was successfully uploaded and executed by the browser. This demonstrates that the application does not adequately validate or restrict uploaded file types, allowing potentially malicious content to be stored and served to users.

Background:

Attackers can exploit file upload vulnerabilities to upload malicious files containing scripts, malware, or web shells. This can lead to Cross-Site Scripting (XSS), session hijacking, credential theft, website defacement, or remote code execution. If an attacker uploads a server-executable file, they may gain unauthorized access to the underlying system and potentially compromise with the entire application or server.

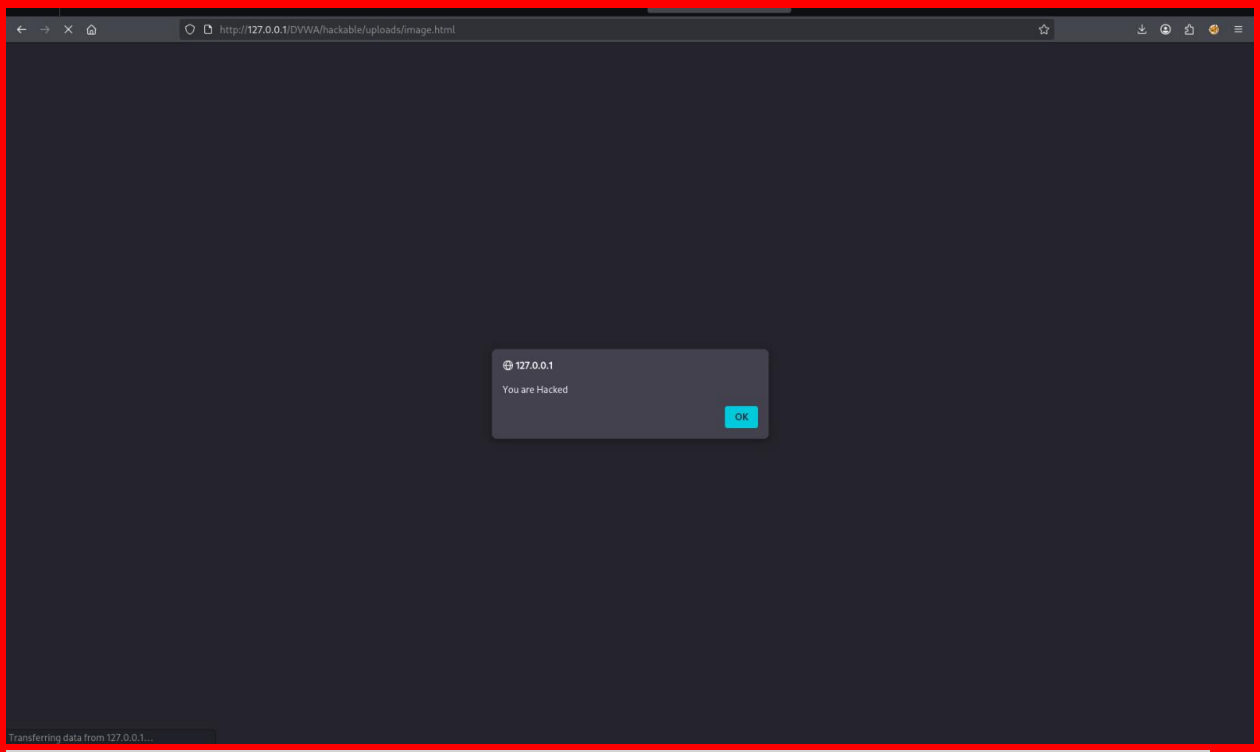
Technical Details and Evidence

During testing, a malicious HTML file was created and uploaded through the application file to upload functionality. The file contained a JavaScript payload:

<script>alert ("You are hacked") </script>







After the file was uploaded and accessed through the application, the script executed successfully within the browser, confirming the presence of a file upload vulnerability that could be leveraged to perform Cross-Site Scripting (XSS) attacks. This demonstrates that the application does not adequately validate or restrict uploaded file content before making it accessible to users.

Impact

A successful file upload attack can allow attackers to upload and execute malicious content within the application. This may lead to Cross-Site Scripting (XSS), session hijacking, credential theft, website defacement, and unauthorized access to user

accounts. In more severe cases, attackers may upload executable files that could result in remote code execution and full server compromise.

Remediation Advice

Implement strict server-side validation of uploaded files, including file type, extension, and content verification. Restrict uploads to approved file formats, store uploaded files outside the web root and disable script execution within upload directories. Additionally, enforce file size limits, rename uploaded files, and regularly scan uploaded content for malicious code.

Title: SQL Injection (T1078 Valid Accounts) reveal sensitive data

Risk rating: Critical

Summary:

A SQL Injection vulnerability was identified during testing of the application. Burp Suite was used to intercept and modify requests sent to the server. Initial testing with a single quotation mark (') triggered an internal server error, indicating improper handling of user input within database queries. Further testing with the payload ' OR 1=1 # successfully manipulated the application's SQL query, confirming the presence of a SQL Injection vulnerability.

Background:

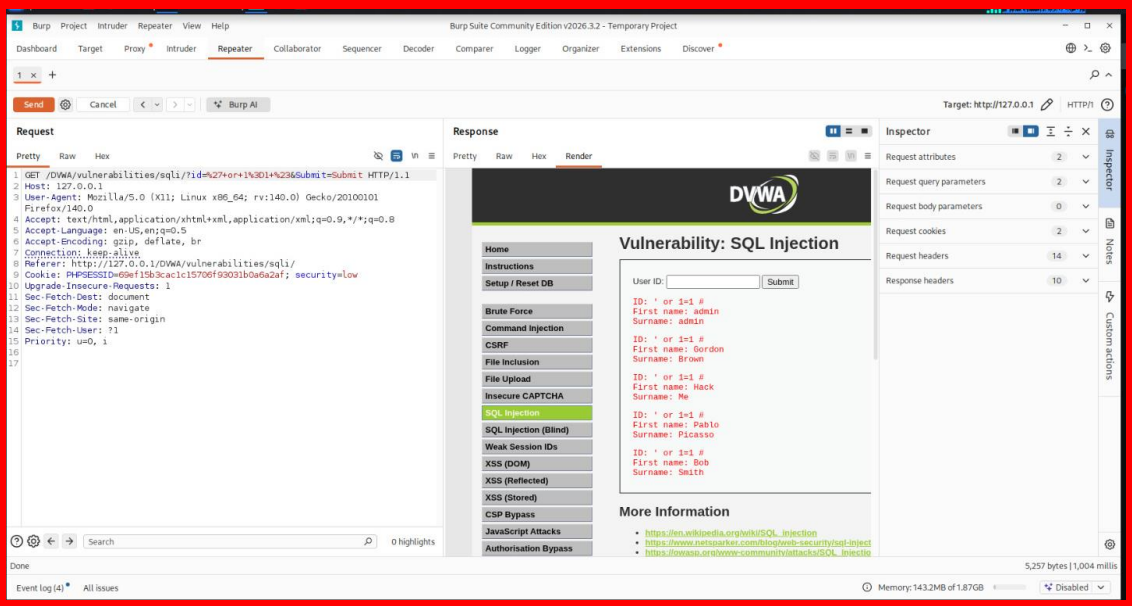
Attackers can exploit SQL Injection vulnerabilities to bypass authentication controls, access sensitive database information, extract user credentials, and modify or delete data. In severe cases, attackers may gain administrative access to the application, compromise the entire database, and use the exposed information to conduct further attacks against users or connected systems.

Technical findings and Evidence:

During testing, an initial SQL Injection probe was performed by submitting a single quotation mark (') as input. The application responded with an **I**nternal Server Error (500), indicating that user-supplied input was not being properly handled within the backend SQL query. This suggested the presence of a potential SQL Injection vulnerability.

Further testing was conducted using various SQL Injection payloads, including ' OR 1=1 --. After multiple attempts, the payload ' OR 1=1 # successfully manipulated the SQL query and caused the application to return sensitive information. This confirmed that the

application was vulnerable to SQL Injection due to insufficient input validation and insecure database query construction.



Impact:

SQL Injection can allow attackers to bypass authentication controls, access sensitive database information, and manipulate application data. Successful exploitation may result in the disclosure of confidential information, unauthorized modification or deletion of records, and in severe cases, complete compromise of the application database.

Remediation advises:

Use parameterized queries (prepared statements) for all database interactions and avoid dynamically constructing SQL queries with user input. Implement strict input validation, apply the principle of least privilege to database accounts, and ensure detailed error messages are not exposed to users. Regular security testing and code reviews should also be performed to identify and remediate SQL Injection vulnerabilities.

Title: Document Object Model based XSS (T1059 Command and Scripting Interpreter)

Risk rating: High

Summary:

A DOM-Based Cross-Site Scripting (XSS) vulnerability was identified within the application. Unlike traditional XSS vulnerabilities, DOM XSS occurs entirely on the client

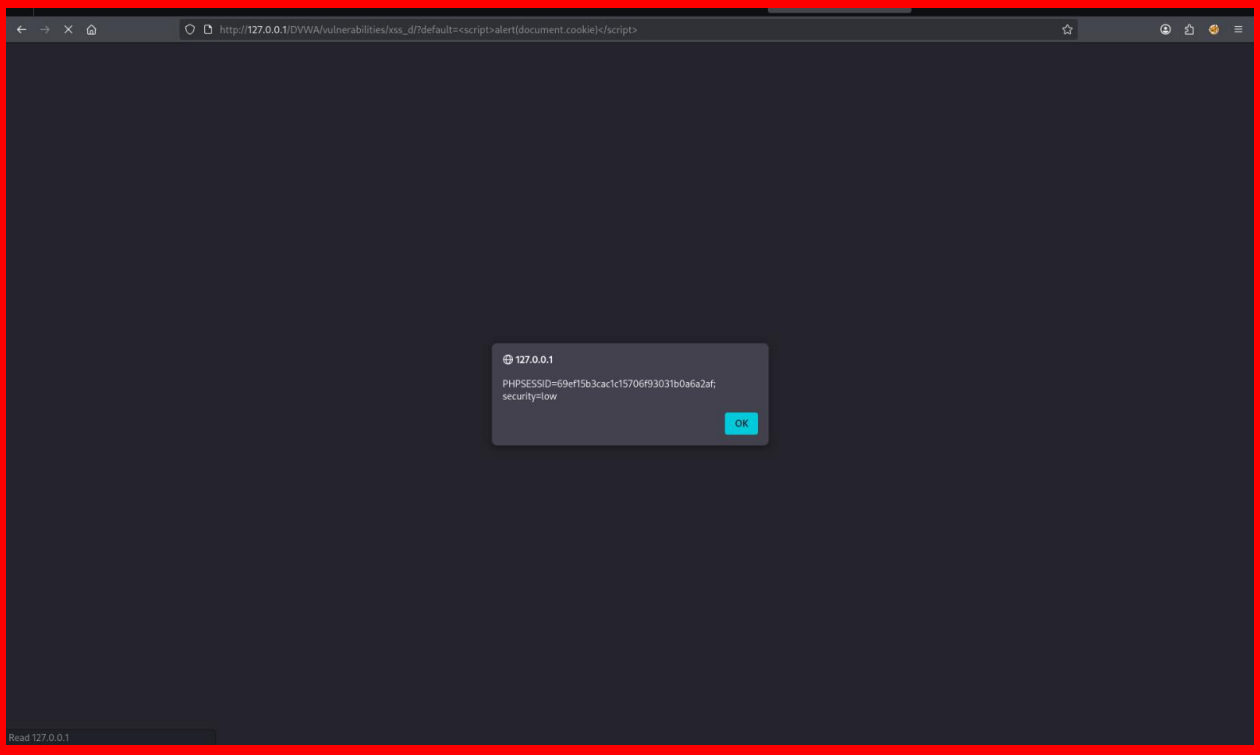
side when JavaScript processes untrusted user input and dynamically updates the Document Object Model (DOM). During testing, the URL was modified to inject a JavaScript payload that executed document.cookie, resulting in the disclosure of browser cookies. This confirmed that user-controlled input was being processed without proper sanitization.

Background:

In a real-world scenario, attackers can exploit DOM-Based XSS by crafting malicious links containing JavaScript payloads and tricking users into visiting them. Successful exploitation can allow attackers to steal session cookies, impersonate users, capture sensitive information, modify webpage content, or perform actions on behalf of the victim. If administrative users are affected, attackers may gain unauthorized access to privileged functionality and sensitive application data.

Technical Details and Evidence:

During testing, the URL was modified by injecting a payload <script>alert(document.cookie)</script>, which executed in the browser and displayed session cookie information. This confirmed that the application processes user-controlled input insecurely on the client side without requiring interaction with the server.



Impact:

The impact of this vulnerability is significant because an attacker can execute arbitrary JavaScript in a victim's browser. This can lead to session hijacking through cookie theft, account impersonation, data exposure, and unauthorized actions performed on behalf of the user. If an administrator is targeted, it could result in full compromise of application functionality and sensitive data.

Remediation advises:

To mitigate this issue, all user input should be properly sanitized and encoded before being rendered in the DOM. Developers should avoid using unsafe JavaScript functions such as inner HTML with untrusted data and instead use secure methods like text Content. Implementing Content Security Policy (CSP) headers can further reduce the risk by restricting execution of unauthorized scripts. Additionally, input validation and context-aware output encoding should be consistently applied across the application to prevent client-side injection attacks.

Title: Insecure token validation (T1552 Unsecured Credentials)

Risk rating: High

Summary:

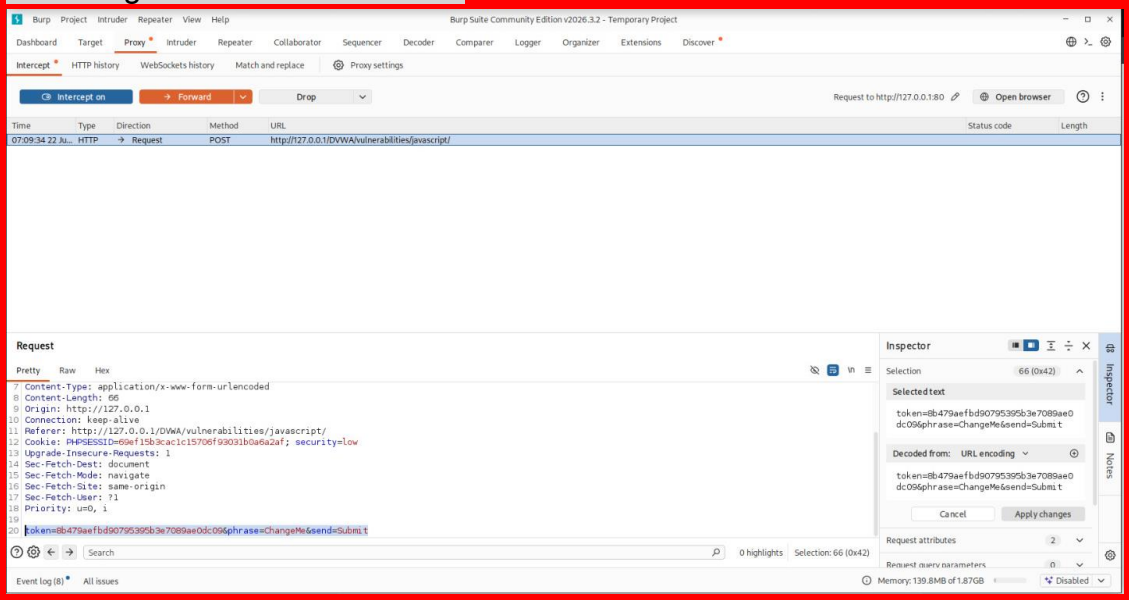
A vulnerability was identified in the client-side JavaScript logic where token validation is handled insecurely within the browser. The token remained static and did not change dynamically, indicating that authentication or validation was being performed on the client side rather than securely on the server. By reviewing the page source, it was observed that the token generation logic involved ROT13 encoding followed by MD5 hashing. Using tools such as CyberChef, it was possible to replicate this process and generate a valid token (for example, by encoding the word "success" and hashing it with MD5), which allowed bypassing the intended validation mechanism.

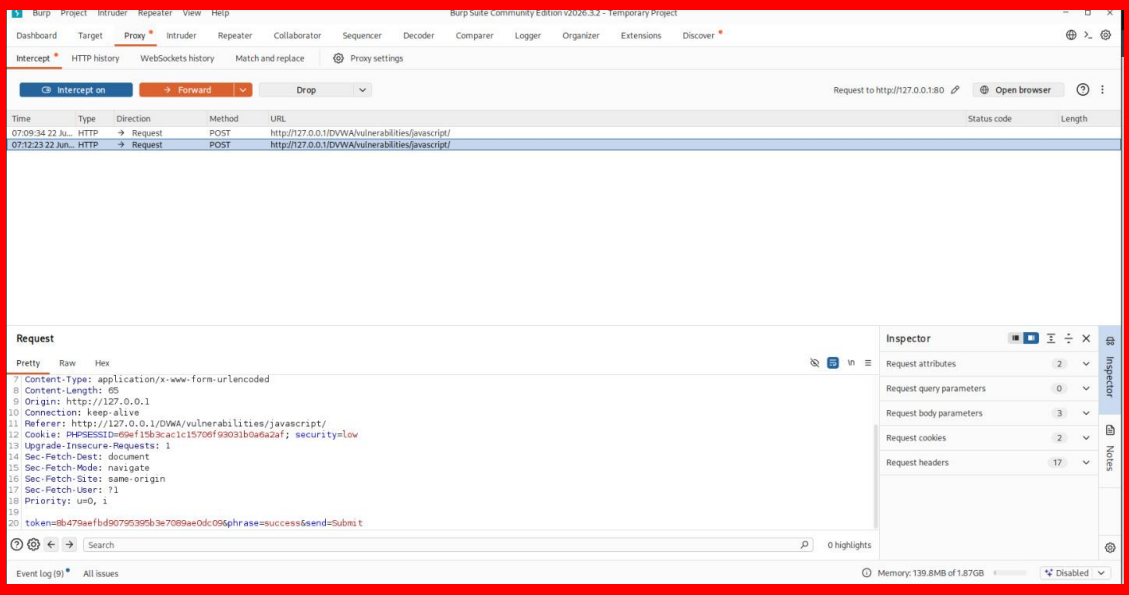
Background:

An attacker can exploit this type of weakness by analyzing client-side code to understand how security controls are implemented and then replicating or manipulating those controls. Since client-side logic is fully visible and under user control, attackers can reverse-engineer encoding or hashing mechanisms to forge valid tokens, bypass authentication checks, or unlock restricted functionality. This can lead to unauthorized access, privilege bypass, and manipulation of application workflows without interacting with the server's true security logic.

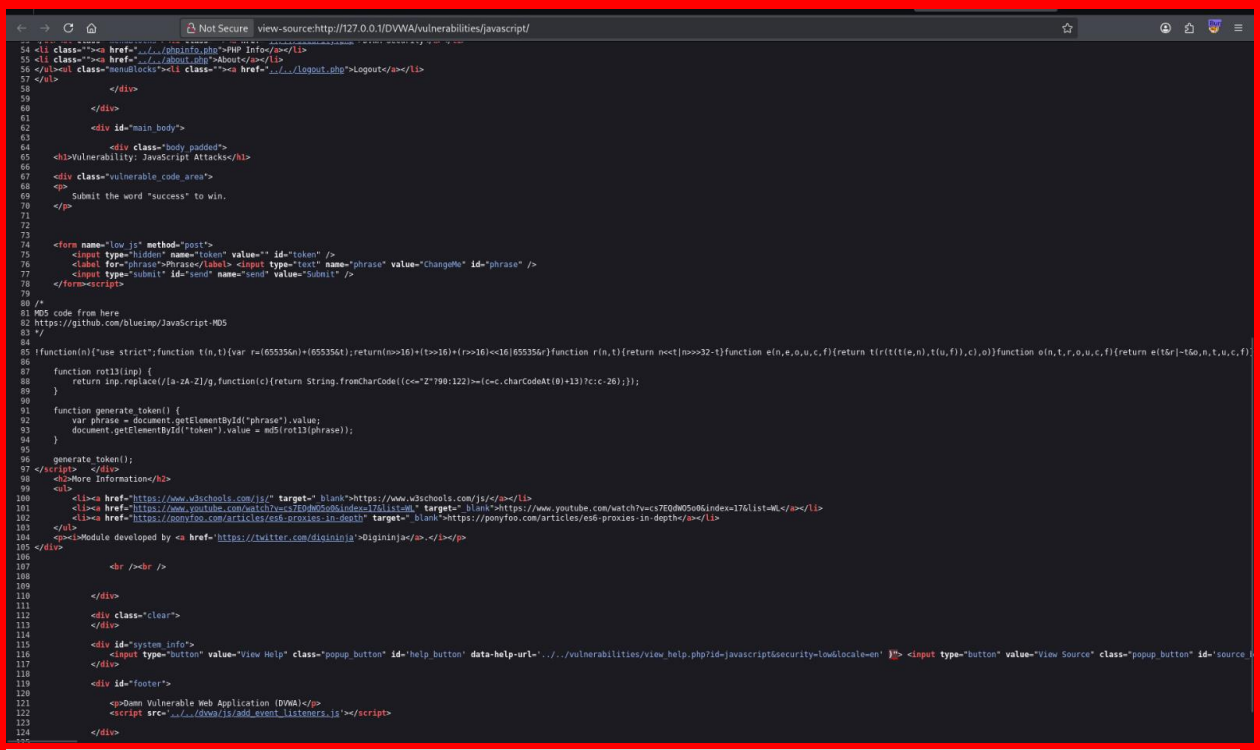
Technical Details and Evidence:

Two POST requests were sent to the server and intercepted using Burp Suite. It was observed that the application reused the same token across multiple requests, indicating weak token validation.

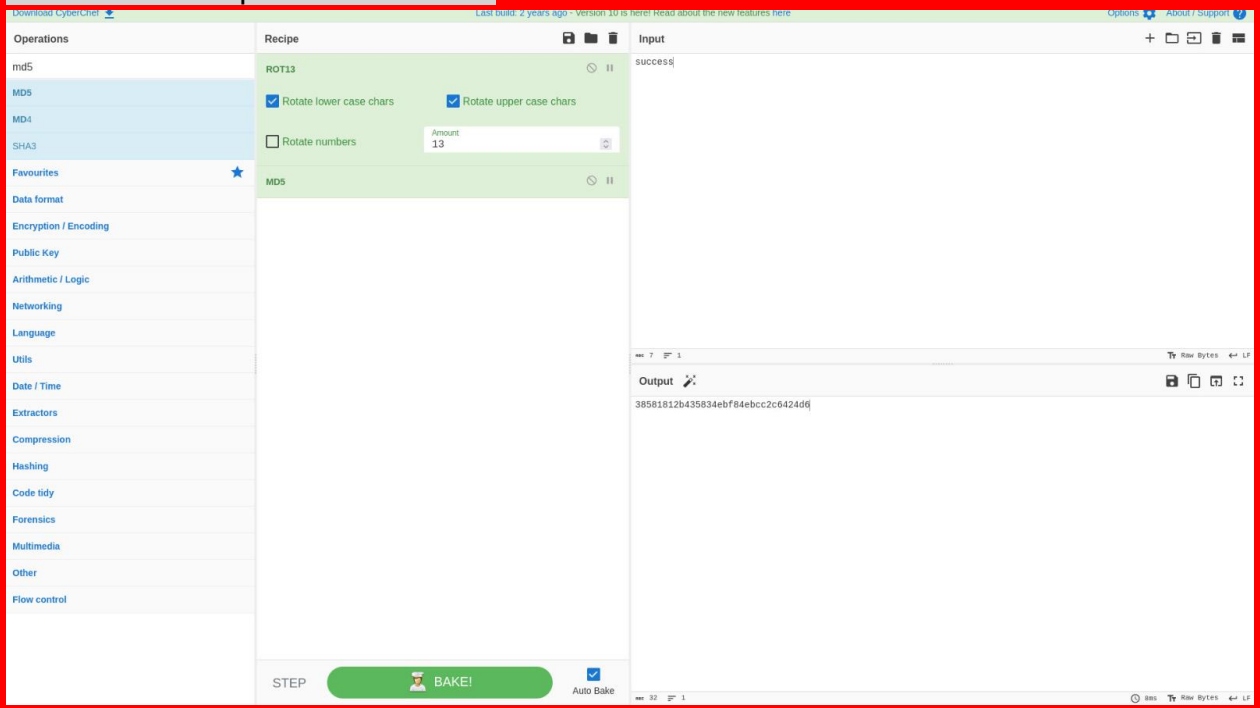




Upon reviewing the page source, the token generation logic was found to involve ROT13 encoding followed by MD5 hashing.

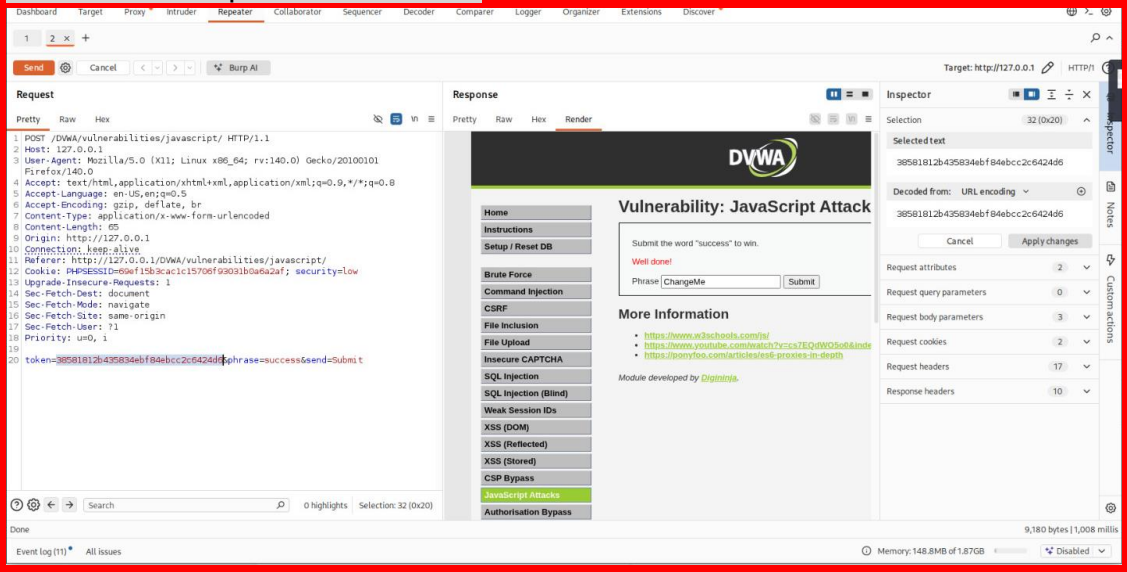


Then to submit the word “success”.Cyberchef was used to encode the word into strings of numbers to represent the token.



The generated token was then inserted into the intercepted request within Burp Suite, and the modified request was forwarded to the server. The application responded with an HTTP 200 status code, confirming that the request was successfully accepted using

the reused and predictable token.



Impact:

The impact of this weakness is significant because predictable or reusable tokens can allow attackers to bypass security controls and perform unauthorized actions. If a token can be generated or reused without proper server-side validation, an attacker can intercept requests, replicate valid tokens, and gain access to restricted functionality. This can lead to authentication bypass, privilege escalation, unauthorized data manipulation, and replay attacks where previously captured requests are reused to trigger valid actions without proper authorization.

Remediation advise:

To remediate this issue, token generation and validation must be fully implemented on the server-side using cryptographically secure random values that are unique per session and per request. Tokens should be unpredictable, time-bound, and invalidated after use to prevent reuse. Client-side logic should never be relied upon for security decisions or token generation. Additionally, proper session management, secure storage of tokens, and enforcement of strict server-side verification for every sensitive request should be implemented to prevent tampering and replay attacks.